ML Problem Solving: K-Means and Gaussian Mixture Models (paper + code)

Overview

In this problem-solving, we will explore two popular clustering methods, performing pen-and-paper calculations using different metrics and implement the methods from scratch using numpy.

Problem Breakdown

Module	Topic	Core Focus
Module 1	K-Means Coordinate Optimization and Hand Execution	Vector quantization formulation, Lloyd-Forgy alternating descent, squared Euclidean distances, and convergence under different initializations
Module 2	Cluster Validation Metrics and Ground-Truth Alignment	Within-Cluster Sum of Squares (WSS), simplified silhouette coefficient, label permutation matching, and classification vs clustering objectives
Module 3	Gaussian Mixture Models and Expectation-Maximization	Convex combination of Gaussians, soft responsibilities (E-step), closed-form parameter updates (M-step), and log-likelihood tracking
Module 4	Vectorized Implementation of K-Means from Scratch	Matrix broadcasting distances, K-means++ initialization, empty cluster handling, and elbow/silhouette diagnostics
Module 5	Vectorized Implementation of GMM from Scratch and Comparative Study	Log-sum-exp numerical stabilization, covariance regularization, EM convergence, model selection (AIC/BIC), and inductive biases

Coding instructions (modules 4 and 5)

Dependencies Allowed for Implementation: `numpy`, `scipy` only (no `scikit-learn` or high-level clustering libraries)

Suggested Benchmark Datasets: A 2D synthetic Gaussian Mixture dataset ($N = 600$ to 1000 samples) with known ground-truth components (spherical, diagonal, and full/anisotropic covariances), alongside the classic Old Faithful Geyser dataset ($N = 272$).

Module 1: K-Means Coordinate Optimization and Hand Execution

Conceptual Background 1: The K-Means Optimization Problem

Given an unlabeled dataset $D = \{x_1, x_2, \ldots, x_N\} \subset \mathbb{R}^d$, the goal of K-means clustering is to partition the N observations into K disjoint clusters $C = \{C_1, C_2, \ldots, C_K\}$ such that the Within-Cluster Sum of Squares (WSS), also called inertia or distortion, is minimized:

$$\text{WSS} = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|_2^2$$

where $\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$ is the empirical centroid of cluster C_k.

The standard Lloyd-Forgy algorithm alternates between two coordinate descent steps:

1. **Assignment Step:** Assign each observation to the nearest centroid under Euclidean distance:

$$c_i = \arg \min_{k \in \{1,\ldots,K\}} \|x_i - \mu_k\|_2^2$$

Ties are broken consistently (e.g., by lowest cluster index).

2. **Update Step:** Recompute the centroid of each cluster:

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

Task 1: Coordinate Geometry and Iterative Execution by Hand

Consider the 5-point dataset:

$$D = [x_1, x_2, x_3, x_4, x_5]^T = \begin{bmatrix} 3 & 3 \\ 1 & 1 \\ -1 & 0 \\ 2 & 2 \\ -2 & 2 \end{bmatrix}$$

1. Sketch or plot the 5 points in a 2D Cartesian plane.
2. Run Lloyd's algorithm by hand for $K = 2$ under two independent centroid initializations until convergence:
   - **Initialization A:** $\mu_1^{(0)} = [3, 3]^T$, $\mu_2^{(0)} = [0, 0]^T$
   - **Initialization B:** $\mu_1^{(0)} = [2, 2]^T$, $\mu_2^{(0)} = [-2, -2]^T$ *(If equidistant to both centroids, assign to the centroid with the lowest index).*
3. For each iteration of both initializations, report:
   - Squared Euclidean distance matrix between all points and current centroids.
   - Cluster assignment vector $c \in \{1, 2\}^5$.
   - Updated centroids μ_1, μ_2.
   - Number of iterations required to reach strict convergence ($\mu^{(t+1)} = \mu^{(t)}$).

Checklist

- Explicit distance calculations shown for every point-to-centroid pair at each iteration.
- Clearly tabulated cluster assignment vectors at each step.
- Final coordinates of converged centroids expressed in exact fractions and decimal approximations.
- Number of iterations to convergence explicitly identified for both initializations.

Module 2: Cluster Validation Metrics and Ground-Truth Alignment

Conceptual Background 2: Evaluation Metrics (WSS, Silhouette, and Classification Alignment)

1. **Within-Cluster Sum of Squares (WSS):** Measures geometric compactness. Lower values indicate tighter clusters:

$$\text{WSS} = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|_2^2$$

2. **Simplified Silhouette Coefficient:** For point $x_i \in C_k$, let $a'_i = \|x_i - \mu_k\|_2$ be the Euclidean distance to its own cluster centroid, and $b'_i = \frac{1}{K-1} \sum_{j \neq k} \|x_i - \mu_j\|_2$ be the average Euclidean distance to all other cluster centroids. The point silhouette is defined as:

$$s'_i = \frac{b'_i - a'_i}{\max(a'_i, b'_i)}$$

The global simplified silhouette is either the sum $\sum_{i=1}^{N} s'_i$ or the sample mean $\frac{1}{N} \sum_{i=1}^{N} s'_i \in [-1, 1]$. Higher values indicate superior separation.

3. **Clustering vs. Classification Accuracy:** When ground truth labels $y_i \in \{0, 1, \ldots, K-1\}$ exist, unsupervised cluster indices c_i have arbitrary label permutations π. The clustering accuracy is defined via the optimal bijective mapping:

$$\text{Acc} = \max_{\pi \in \text{Perm}(K)} \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}(y_i = \pi(c_i))$$

Task 2: Cluster Validation Metrics and Ground-Truth Evaluation

Using the converged cluster assignments from Question 1:

1. **WSS Computation:** Compute the exact Within-Cluster Sum of Squares (WSS) for both converged partitions. State which initialization achieved the better compactness.
2. **Simplified Silhouette Computation:** For every point x_i ($i = 1, \ldots, 5$), compute a'_i, b'_i, and s'_i under both converged solutions. Compute the total sum $\sum_{i=1}^{5} s'_i$ and the mean simplified silhouette. State which initialization achieved the better silhouette.
3. **Supervised Alignment:** The dataset is augmented with true class labels $y \in \{0, 1\}$:

$$D_{\text{labeled}} = \begin{bmatrix} 3 & 3 & 1 \\ 1 & 1 & 1 \\ -1 & 0 & 1 \\ 2 & 2 & 0 \\ -2 & 2 & 0 \end{bmatrix}$$

- Evaluate the optimal clustering classification accuracy for Initialization A and Initialization B.
- Provide an analytical discussion explaining how clustering accuracy relates to WSS and the Silhouette metric in this problem, and why a lower WSS does not guarantee higher classification accuracy.

Checklist

- Step-by-step arithmetic breakdown of squared deviations per cluster for WSS.
- Table listing i, x_i, a'_i, b'_i, s'_i for all 5 points under both configurations.
- Clear best-case determination for both WSS and Silhouette metrics.
- Explicit label permutation matching and accuracy computation.
- A concise conceptual paragraph explaining the fundamental mismatch between unsupervised geometric cost functions and supervised classification objectives.

Module 3: Gaussian Mixture Models and EM Pen-and-Paper Calculations

Conceptual Background 3: Gaussian Mixture Formulation and EM

A Gaussian Mixture Model represents a continuous probability distribution as a convex combination of K multivariate Gaussian components:

$$p(x \mid \theta) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)$$

where: * $\pi_k \in [0,1]$ are mixing coefficients satisfying $\sum_{k=1}^{K} \pi_k = 1$. * $\mu_k \in \mathbb{R}^d$ is the mean vector of component k. * $\Sigma_k \in \mathbb{R}^{d \times d}$ is the symmetric positive-definite covariance matrix. * In $d = 1$, $\mathcal{N}(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$.

The marginal log-likelihood of dataset $X = \{x_1, \ldots, x_N\}$ is:

$$\ln p(X \mid \theta) = \sum_{i=1}^{N} \ln \left(\sum_{k=1}^{K} \pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k) \right)$$

Because direct maximization of $\ln p(X \mid \theta)$ contains a summation inside the logarithm, optimization is performed via the Expectation-Maximization (EM) algorithm using latent component indicator variables $z_i \in \{0,1\}^K$:

1. **E-Step (Expectation):** Evaluate posterior probabilities (responsibilities):

$$\gamma_{ik} = P(z_{ik} = 1 \mid x_i, \theta) = \frac{\pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_i \mid \mu_j, \Sigma_j)}$$

2. **M-Step (Maximization):** Update parameters:

$$N_k = \sum_{i=1}^{N} \gamma_{ik}, \quad \pi_k^{\text{new}} = \frac{N_k}{N}, \quad \mu_k^{\text{new}} = \frac{1}{N_k} \sum_{i=1}^{N} \gamma_{ik} x_i, \quad \Sigma_k^{\text{new}} = \frac{1}{N_k} \sum_{i=1}^{N} \gamma_{ik} (x_i - \mu_k^{\text{new}})(x_i - \mu_k^{\text{new}})^T$$

3. **Connection to K-Means:** K-means is the zero-variance, equal-prior deterministic limit of GMM. If $\Sigma_k = \epsilon I$ for all k and $\pi_k = \frac{1}{K}$, taking the limit $\epsilon \to 0$ collapses the soft responsibilities γ_{ik} into hard 1-of-K indicator assignments.

Task 3: Step-by-Step EM Derivation on a 1D Dataset

Consider a 1D dataset of $N = 3$ scalar points:

$$X = \{x_1 = 0.0, \quad x_2 = 2.0, \quad x_3 = 6.0\}$$

We model X with a 2-component Gaussian mixture ($K = 2$) initialized with equal mixing priors and unit variances: * $\pi_1^{(0)} = 0.5, \quad \mu_1^{(0)} = 1.0, \quad (\sigma_1^2)^{(0)} = 1.0$ * $\pi_2^{(0)} = 0.5, \quad \mu_2^{(0)} = 5.0, \quad (\sigma_2^2)^{(0)} = 1.0$

Given reference constants $\frac{1}{\sqrt{2\pi}} \approx 0.3989$, $\exp(-0.5) \approx 0.6065$, $\exp(-4.5) \approx 0.0111$, $\exp(-8.0) \approx 0.000335$, and $\exp(-12.5) \approx 0.0000037$:

1. Compute the component Gaussian densities $\mathcal{N}(x_i \mid \mu_k, \sigma_k^2)$ for all $i \in \{1,2,3\}$ and $k \in \{1,2\}$.
2. Execute the **E-Step**: Compute the 3×2 responsibility matrix $\Gamma = [\gamma_{ik}]$.
3. Execute the **M-Step**: Compute effective cluster counts N_1, N_2; Compute updated mixing proportions $\pi_1^{(1)}, \pi_2^{(1)}$; Compute updated component means $\mu_1^{(1)}, \mu_2^{(1)}$; Compute updated component variances $(\sigma_1^2)^{(1)}, (\sigma_2^2)^{(1)}$.
4. Compute the marginal log-likelihood before the update ($\ln p(X \mid \theta^{(0)})$) and verify that the objective function increased after the M-step.

Checklist

- Explicit probability density value evaluated for each sample and component.
- Complete normalized responsibility matrix where rows sum to 1.000.
- Step-by-step arithmetic for updated parameters $\pi_k^{(1)}, \mu_k^{(1)}, (\sigma_k^2)^{(1)}$.
- Exact log-likelihood expression with intermediate numerical sums.

Module 4: Vectorized Implementation of K-Means from Scratch

Conceptual Background 4: Vectorized Distance Computation and Initialization

To scale K-means to larger sample sizes without slow Python loops over samples, distances must be vectorized using linear algebra identities. Let $X \in \mathbb{R}^{N \times d}$ and $M \in \mathbb{R}^{K \times d}$. The squared Euclidean distance matrix $D \in \mathbb{R}^{N \times K}$ satisfies:

$$\|x_i - \mu_k\|_2^2 = \|x_i\|_2^2 + \|\mu_k\|_2^2 - 2x_i^T \mu_k$$

In NumPy broadcasting: `D = np.sum(X**2, axis=1, keepdims=True) + np.sum(M**2, axis=1) - 2 * np.dot(X, M.T)`

Initialization Strategies: 1. **Random Forgy:** Select K unique samples uniformly at random from X. 2. **K-Means++:** * Choose first centroid μ_1 uniformly at random from X. * For $k = 2, \ldots, K$, select $\mu_k = x_i$ with probability proportional to $D(x_i)^2 = \min_{j<k} \|x_i - \mu_j\|_2^2$.

Task 4: K-Means Implementation, Benchmarking, and Diagnostics

Using only `numpy` and `scipy`:

1. Implement a class `KMeansScratch` with the following interface:
   - `__init__(self, n_clusters=2, init='k-means++', max_iter=300, tol=1e-4, random_state=None)`
   - `fit(self, X)`: Alternates between vectorized assignment and centroid recalculation. Terminates when centroid shift $\|\mu^{(t+1)} - \mu^{(t)}\|_F^2 <$ tol or `max_iter` is reached. Handles empty clusters by reinitializing the empty centroid to the data point with the highest residual error.
   - `predict(self, X)`: Returns cluster indices for input data.
   - `compute_wss(self, X)`: Returns the final inertia / WSS.
   - `compute_simplified_silhouette(self, X)`: Computes the sample-averaged simplified silhouette score.
2. **Benchmark Dataset Specification:**
   - Generate a 2D synthetic dataset $X_{\text{bench}} \in \mathbb{R}^{800 \times 2}$ consisting of 3 distinct clusters with ground-truth assignments:
     - Component 1: $N_1 = 300$, $\mu = [2.0, 2.0]$, isotropic covariance $\Sigma_1 = 0.4 \cdot I_2$.
     - Component 2: $N_2 = 300$, $\mu = [-2.0, -2.0]$, isotropic covariance $\Sigma_2 = 0.5 \cdot I_2$.
     - Component 3: $N_3 = 200$, $\mu = [3.0, -3.0]$, isotropic covariance $\Sigma_3 = 0.3 \cdot I_2$.
3. Run `KMeansScratch` on X_{bench} across $K \in \{2, 3, 4, 5, 6\}$.
   - Track WSS and average Simplified Silhouette across iterations.
   - Verify whether the elbow in WSS and the peak in Silhouette correctly identify the true cluster count $K = 3$.

Checklist

- Python code adhering to vectorized NumPy paradigms (no nested loops over data samples).
- Proper implementation of both `'random'` and `'k-means++'` initialization schemes.
- Correct handling of potential empty clusters during execution.
- Tabulated execution summary over $K \in \{2, 3, 4, 5, 6\}$ listing WSS, Simplified Silhouette, and number of iterations to convergence.

Module 5: Vectorized Implementation of GMM from Scratch and Comparative Study

Conceptual Background 5: Numerical Stability and Model Selection

Direct computation of Gaussian probabilities in high dimensions or across multiple iterations often leads to floating-point underflow or degenerate singular matrices. Two engineering safeguards are required:

1. **Covariance Regularization:** Add a small positive constant to the diagonal to guarantee positive-definiteness:

$$\Sigma_k \leftarrow \Sigma_k + \epsilon_{\text{reg}} I_d, \quad \epsilon_{\text{reg}} = 10^{-6}$$

2. **Log-Domain Responsibility Computation:** Evaluate log-densities $\ln \mathcal{N}(x_i \mid \mu_k, \Sigma_k)$ directly using `scipy.stats.multivariate_normal.logpdf`. The responsibility equation is transformed via:

$$\ln \gamma_{ik} = \ln \pi_k + \ln \mathcal{N}(x_i \mid \mu_k, \Sigma_k) - \text{logsumexp}_j \left(\ln \pi_j + \ln \mathcal{N}(x_i \mid \mu_j, \Sigma_j) \right)$$

where $\text{logsumexp}(v) = \max(v) + \ln \left(\sum_j \exp(v_j - \max(v)) \right)$.

3. **Model Selection Criteria (AIC and BIC):** Given p free parameters:

$$p = (K-1) + K \cdot d + K \cdot \frac{d(d+1)}{2}$$

- **Bayesian Information Criterion (BIC):** $\text{BIC} = -2 \ln p(X \mid \hat{\theta}) + p \ln N$
- **Akaike Information Criterion (AIC):** $\text{AIC} = -2 \ln p(X \mid \hat{\theta}) + 2p$

Task 5: GMM Implementation, Convergence, and Model Selection

Using only `numpy` and `scipy`:

1. Implement a class `GaussianMixtureScratch` with the following specification:
   - `__init__(self, n_components=2, max_iter=200, tol=1e-4, reg_covar=1e-6, random_state=None)`
   - Centroid initialization via a preliminary run of K-means or K-means++ to prevent slow convergence.
   - `fit(self, X)`: Executes vectorized E-step and M-step. Computes log-likelihood at every iteration and checks convergence via $|\ln p(X \mid \theta^{(t)}) - \ln p(X \mid \theta^{(t-1)})| < \text{tol}$.
   - `predict_proba(self, X)`: Returns posterior responsibility matrix $\Gamma \in [0,1]^{N \times K}$.
   - `predict(self, X)`: Returns hard assignments $\arg\max_k \gamma_{ik}$.
   - `bic(self, X)` and `aic(self, X)`: Implements parameter counting and information criteria.
2. **Anisotropic Experimentation:** Generate an anisotropic dataset $X_{\text{aniso}} \in \mathbb{R}^{600 \times 2}$ composed of two elongated, overlapping elliptical clusters:
   - Component 1: $N_1 = 300$, $\mu_1 = [0.0, 0.0]$, $\Sigma_1 = \begin{bmatrix} 2.0 & 1.5 \\ 1.5 & 1.5 \end{bmatrix}$.
   - Component 2: $N_2 = 300$, $\mu_2 = [4.0, 4.0]$, $\Sigma_2 = \begin{bmatrix} 1.8 & -1.2 \\ -1.2 & 1.2 \end{bmatrix}$.
3. Compare the clustering behavior of K-Means vs. Gaussian Mixture Model on X_{aniso}:
   - Explain why K-means fails to recover the true underlying cluster boundaries on this dataset.
   - Run GMM with $K \in \{1, 2, 3, 4, 5\}$ and verify whether BIC attains its global minimum at $K = 2$.

Checklist

- Complete object-oriented implementation with numerical stability guards.
- Explicit log-likelihood monotonicity tracking across iterations.
- Accurate parameter counting and BIC/AIC computation.
- Detailed written explanation contrasting the inductive biases of K-means (spherical, equal variance) against full-covariance GMM (arbitrary orientation and eccentricities).